CONSTRUCTION GUIDE

CV Pipeline Talend

Building an interactive resume driven by a Talend job
Serverless architecture, data in JavaScript, editing interface.

Technical guide — reproducible with Talend Open Studio for Data Integration (Community edition)

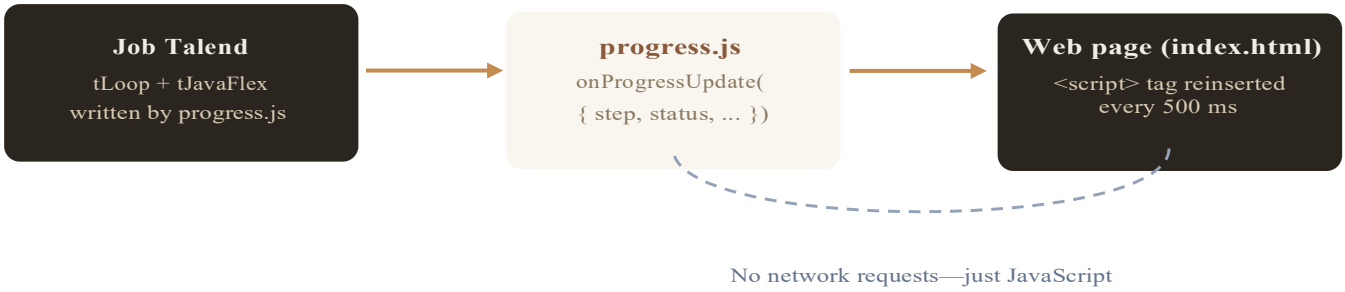
Table of Contents

1. Architecture Overview
2. Prerequisites
3. Step 1 — Creating the Project and Data (.js)
4. Step 2 — The Web Page (index.html)
5. Step 3 — The Style Sheet (cream/terracotta design)
6. Step 4 — The app.js script
7. Step 5 — Building the Talend job component by component
8. Step 6 — Run and test
9. Step 7 (optional) — Resume upload section
10. Final project directory structure
11. Verification checklist

> All examples of profiles, skills, and experience in this guide are fictional (candidate “Han Onym”).
> Replace them with your own information in the files in the DATA/ folder.

1. Architecture Overview

Layer	Role	Technology
TALEND	The engine orchestrates the steps and dictates the progression	Talend Open Studio (job)
DATA	The data in the CV is separated from the code	Files .js (JavaScript objects)



2. Prerequisites

- Talend Open Studio for Data Integration (Community version)
- A recent web browser (Chrome, Edge, Firefox)
- A text editor (VS Code, Notepad++ or equivalent)
- No server, no additional runtime

3. Step 1 – Project and Data Creation (.js)

1 Create the following directory structure on your hard drive, for example in ../CV_PIPELINE_TALEND :

```
CV_TALEND/
├── DATA/
| ├── profile.js
| ├── skills.js
| ├── experiences.js
|└── education.js
├── WEB/
| ├── index.html
| ├── css/style.css
|└── js/app.js
├── RUNTIME/
|└── progress.js
└── TALEND/   (Your job will be created here in Talend Studio)
```

Each file in the `DATA/` directory is a JavaScript object assigned to a global property, which is then loaded by a `<script>` tag in `index.html`. Editing rule: Never change the first line or the closing semicolon; modify only the content between the curly braces.

DATA/profile.js

```
window.CV_DATA = window.CV_DATA || {};
window.CV_DATA.profile = {
  "firstname": "Han",
  "lastname": "Onym",
  "title": " Fire Chief ",
  "email": "camille.rocher@exemple.com",
  "phone": "06.00.00.00.00",
  "linkedin": "linkedin.com/in/han-onym",
  "summary": "rescue record holder" };
```

DATA/skills.js

```
window.CV_DATA = window.CV_DATA || {};
window.CV_DATA.skills = {
  "skills": [
    { "category": "veritatis ", "items": ["righteous"] },
    { "category": "inventore ", "items": ["recusandae"] },
    { "category": "bonorum", "items": ["dignissimos "] }
  ]
};
```

DATA/experiences.js (fictitious data)

```
window.CV_DATA = window.CV_DATA || {};
window.CV_DATA.experiences = {
  "experiences": [
    {
      "period": "2023 → 2025",
      "client": "Norduo Retail",
      "role": "asperiores",
      "description": "Et harum quidem rerum facilis est et expedita distinctio.",
      "environment": ["quas"]
    },
    {
      "period": "2020 → 2023",
      "client": "optio cumque",
      "role": "Lead",
      "description": "omnis voluptas assumenda est",
      "environment": ["impedit"]
    }
  ]
};
```

DATA/education.js

```
window.CV_DATA = window.CV_DATA || {};
window.CV_DATA.education = {
  "education": [
    { "year": "2010", "degree": "omnis voluptas", "institution": "University of Cloud" }
  ],
  "notes": ""
};
```

4. Step 2 – Web page (index.html)

The page is divided into three sections: a top bar (title + status), a progress bar, and a two-column section—the left column displays the steps and checks, while the right column shows the resume as it is

being created.

```
<!DOCTYPE html>
<html lang="fr">

<head>

<meta charset="UTF-8">
<title>CV Pipeline</title>

<link rel="stylesheet" href="css/style.css">
</head>

<body>

<div class="wrap">

  <div class="frame">

    <div class="topbar">

      <div  class="brand">CV_PIPELINE_<b>CAMILLE</b></div>

      <div class="engine"><span class="dot" id="statusdot"></span><span id="statustext">En attente...</span></div>
    </div>

    <div class="progressrow">

      <div class="progressbar"><div class="progressfill" id="progressfill"></div></div>

      <div class="progresspct" id="progresspct">0%</div>
    </div>

    <div class="body2">

      <div class="log">

        <div class="logtitle">JOB LOG</div>

        <div class="step" id="s1"><span class="num">01</span> INITIALIZATION</div>

        <div class="step" id="s2"><span class="num">02</span> PROFILE</div>
        <div class="step" id="s3"><span class="num">03</span> SKILLS</div>

        <div class="step" id="s4"><span class="num">04</span> EXPERIENCE</div>
        <div class="step" id="s5"><span class="num">05</span> FINALIZATION</div>


        <div class="logdivider"></div>

        <div class="logtitle">CONTRÔLES</div>
        <div class="ctrlcol">

          <button  class="ctrlbtn"  id="previewbtn">    Aperçucomplet</button>
          <button class="ctrlbtn" id="replaybtn">▶ Replay</button>

          <button class="ctrlbtn" id="pausebtn">II Pause</button>

          <button class="ctrlbtn" id="stepbtn">Step →</button>
        </div>

      </div>


      <div class="content" id="content">

        <div class="cvblock" id="step-1"></div>
        <div class="cvblock" id="step-2"></div>

        <div class="cvblock" id="step-3"></div>

        <div class="cvblock" id="step-4"></div>
        <div class="cvblock" id="step-5"></div>

      </div>
    </div>

  </div>

</div>

<script src="../DATA/profile.js"></script>

<script src="../DATA/skills.js"></script>
<script src="../DATA/experiences.js"></script>

<script src="../DATA/education.js"></script>
<script src="js/app.js"></script>

</body>

</html>
```

5. Step 3 – The style sheet (cream/terracotta design)

Color palette used:

Usage	Couleur	Code
Page Background		#EFE7D8
Map Background		#FAF7F1
Borders		#E4D9C9
Accent (terracotta)		#B5714F
Main Text		#2B2620
Faded Text		#9C8F7C
Step Completed (sage)		#7C8F6E
Pause (amber)		#D9A441

Fonts: Georgia, “Times New Roman” (serif) for the name and experiment titles; “Helvetica Neue” and Arial (sans-serif) for the rest.

The complete stylesheet (structure, sidebar, skill icons, timeline, etc.) is provided as-is in the project's WEB/css/style.css file.

Copy it as is, or adjust only the colors in the table above to customize the color palette.

6. Step 4 – The app.js script

Two distinct responsibilities:

- **Content construction**: Upon loading, `buildContent()` reads `window.CV_DATA` (populated by the `<script>` tags in DATA/) and fills in the five blocks, step-1 through step-5.

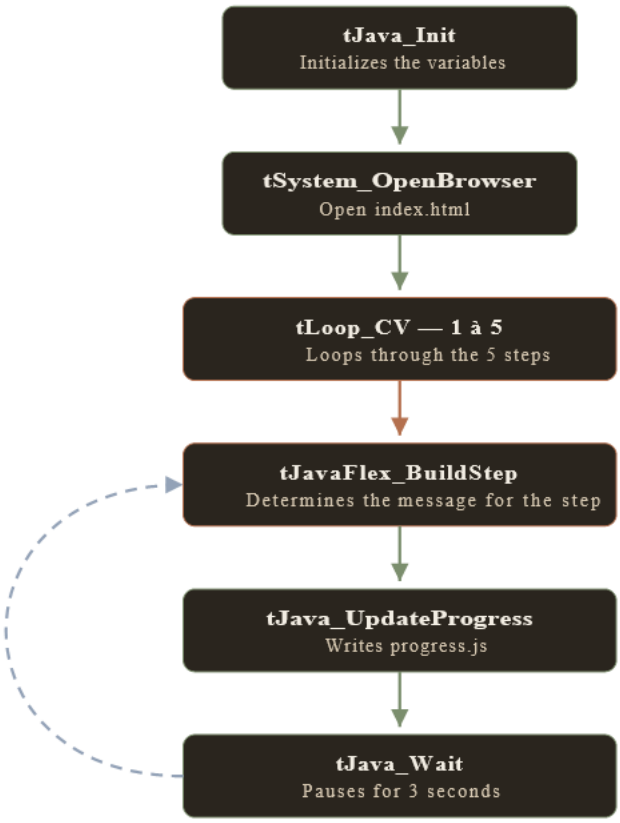
- **Suivi de la progression** : Progress tracking: pollProgress() re-inserts a <script src="progress.js?t=…"> tag every 500 ms. This file calls onProgressUpdate({…}), a global function defined in app.js, which reveals the sections as they become available.

```
const PROGRESS_PATH = "../RUNTIME/progress.js";
let latestFileStep = 0, currentDisplayed = 0, paused = false, livePolling = true;

function buildContent() {
  if (!window.CV_DATA || !window.CV_DATA.profile) { return; }
  renderHeader(window.CV_DATA.profile);
  renderProfile(window.CV_DATA.profile);
  renderSkills(window.CV_DATA.skills);
  renderExperiences(window.CV_DATA.experiences);
  renderEducation(window.CV_DATA.education);
}

let progressScriptTag = null;
window.onProgressUpdate = function (data) {
  latestFileStep = data.step;
  updateStatusBar(data);
  tryReveal();
};

function pollProgress() {
  if (!livePolling) return;
  if (progressScriptTag) progressScriptTag.remove();
  progressScriptTag = document.createElement("script");
  progressScriptTag.src = PROGRESS_PATH + "?t=" + Date.now();
  document.body.appendChild(progressScriptTag);
  setTimeout(pollProgress, 500);
}

function tryReveal() {
  if (paused) return;
  if (currentDisplayed < latestFileStep) {
    currentDisplayed++;
    revealStep(currentDisplayed);
    setTimeout(tryReveal, 400);
  }
}

buildContent();
pollProgress();
```

The complete file (including the `renderHeader`, `renderSkills`, and `renderExperiences` functions, as well as the Replay/Pause/Step/Preview controls, etc.) is provided in `WEB/js/app.js`.
You can copy it in its entirety; it contains no personal data, only generic logic.

> The "Full Overview" button directly calls `onProgressUpdate({step:5, total:5, status:"COMPLETED", ...})`
> and disables polling (`livePolling = false`).
> This allows you to view the final output without launching Talend, which is useful for testing or giving a quick demo.

7. Step 5 – Building a Talend job component by component



Connection Table

Since	Link Type	Toward
tJava_Init	OnSubjobOk	tSystem_OpenBrowser
tSystem_OpenBrowser	OnSubjobOk	tLoop_CV
tLoop_CV	Iterate	tJavaFlex_BuildStep
tJavaFlex_BuildStep	OnComponentOk	tJava_UpdateProgress
tJava_UpdateProgress	OnComponentOk	tJava_Wait
tJava_Wait	(fin d'itération)	retour automatique au tLoop_CV

Variables context

Variable	Exemplary value
context.cv_path	PATH_TO_DIRECTORY/CV_TALEND
context.web_path	PATH_TO_DIRECTORY/CV_TALEND/WEB
context.progress_path	PATH_TO_DIRECTORY/CV_TALEND/RUNTIME/progress.js
context.total_steps	5
context.step_duration_ms	3000

tJava_Init

Resets the step counter

```
globalMap.put("CV_STEP", 0);
globalMap.put("CV_TOTAL_STEPS", context.total_steps);
System.out.println("CV_PIPELINE — INITIALIZATION");
```

tSystem_OpenBrowser

Open the browser once, at WEB/index.html

```
Windows : cmd /c start "" "" + context.web_path + "/index.html"
macOS   : open "" + context.web_path + "/index.html"
Linux   : xdg-open "" + context.web_path + "/index.html"
```

tLoop_CV

Loop type: For — From: 1 — To: context.total_steps — Step: 1

tJavaFlex_BuildStep

Determines the message associated with the current step.
It does not build the CV itself; app.js handles that from DATA/.

```
int step = ((Integer) globalMap.get("CV_STEP")) + 1;
globalMap.put("CV_STEP", step);

String message = "";
switch(step) {
    case 1: message = "Loading candidate profile..."; break;
    case 2: message = "Building profile section..."; break;
    case 3: message = "Processing skill dataset..."; break;
    case 4: message = "Processing professional experience..."; break;
    case 5: message = "Generating final CV..."; break;
}
globalMap.put("CV_MESSAGE", message);
System.out.println("[STEP " + step + "/5] " + message);
```

tJava UpdateProgress

Written in RUNTIME/progress.js — in JSONP format, not raw JSON, so it can be loaded via a <script> tag.

```
int step = ((Integer) globalMap.get("CV_STEP")) + 1;
globalMap.put("CV_STEP", step);

String message = "";
switch(step) {

    case 1: message = "Loading candidate profile...": break;
    case 2: message = "Building profile section...": break;
    case 3: message = "Processing skill dataset...": break;
    case 4: message = "Processing professional experience...": break;
    case 5: message = "Generating final CV...": break;

}
globalMap.put("CV_MESSAGE", message);
System.out.println("[STEP " + step + "/5] " + message);
```

The key point: the .js variable contains a function call (onProgressUpdate({…});), not a raw JSON object ({…}). A raw object loaded as a script causes an “Uncaught SyntaxError: Unexpected token ‘;’” error in the browser console, which is almost certainly a sign that this component is failing just like the old format.

tJava UpdateProgress

Break between two stages

```
Thread.sleep(context.step_duration_ms);
```

8. Step 6 – Launch and Test

1. Double-click WEB/index.html . The page opens immediately, without requiring any installation. As long as no jobs have run, it remains blank (status “Pending”), this is normal.
2. Click “Full Overview” to verify that your data is displayed correctly, regardless of Talend.
3. Run the CV_TALEND job from Talend Studio.

   The Browser opens automatically, and the resume is built section by section

> If the page freezes (the page doesn't update even though the job is running), open the browser console (F12 → Console). In almost all cases, an error will be visible there; this is the quickest way to troubleshoot the issue.

9. Step 7 – (optional) – Resume Download Section

To offer multiple versions of your resume (detailed, simplified, complete package) for download directly from the page, add this block to `index.html`, inside `.log`, right after the `CONTROLS` block:

```
<div class="logdivider"></div>
<div class="logtitle">DOWNLOADS</div>
<div class="ctrlcol">
  <a class="ctrlbtn" href="cv/CV_Detailed.pdf" download>CV Detailed - PDF</a>
  <a class="ctrlbtn" href="cv/CV_Simplified.pdf" download>CV Simplified PDF</a>
  <a class="ctrlbtn" href="cv/Pack_Full_CV.zip" download>Pack Full CV - ZIP</a>
  </div>
```

And in `style.css`, there's just one rule (links don't have the same default style as buttons):

```
a.ctrlbtn { display: block; text-decoration: none; }
```

Create a folder named WEB/cv/ that contains the actual files referenced by these links.

10. Step 8 - Final Project Tree Structure

```
CV_TALEND/
├── DATA/
│   ├── profile.js
│   ├── skills.js
│   ├── experiences.js
│   └── education.js
├── CV/ (optional – PDF/ZIP downloadable)
│   ├── PDF
│   ├──── CV_xxxx.pdf
│   ├── ZIP
│   └──── CV_xxxx.zip
├── DOCUMENTATION/
│   ├── Construction Guide.pdf
│   ├── Architecture Note.pdf
│   ├── README
q├── WEB/
│   ├── index.html
│   ├── CSS
│   ├──── style.css
│   ├── JS
│   ├──── app.js
├── RUNTIME/
│   └── progress.js
└── TALEND/
    └── 00_CV_Pipeline_Talend
    └── 01_tJava_Init
    └── 02_tSystem_OpenBrowser
    └── 03_tLoop_CV
    └── 04_tJavaFlex_BuildStep
    └── 05_tJava_UpdateProgress
    └── 06_tJava_Wait
    └── JOB_CONSTRUCTION
└── README/
```

11. Step 9 - Verification Checklist

- ☐ The 4 .js files are located in DATA/ and contain your actual data
- ☐ index.html successfully loads the 4 files from DATA/ and then js/app.js, in that order
- ☐ .log closes before .content (no accidentally nested sections)
- ☐ The “Full Overview” button correctly displays the entire CV
- ☐ context.progress_path points to RUNTIME/progress.js (not .json), in the correct location
- ☐ tJava_UpdateProgress correctly writes onProgressUpdate({...}); and not a raw JSON object
- ☐ After a Talend run, the page builds section by section with no errors in the console (F12)
- ☐ Tested by double-clicking directly on index.html, without a server